Q・K・V を開ける

自己注意の実装 — 有名な式の正体は行列積と softmax だけ

rkskmt

前回の振り返り

- 注意＝3ステップの表計算：**関連度（内積）→ 割合（softmax）→ 重み付き平均**
- 手計算も検算も済んだ。残った疑問はこれだった：

> **① 前回の宿題**
>
> 名札 k・持ってる情報 v・探しもの q の中身は、**誰が書くのか？**

- 答え：文字の埋め込みベクトルから、**行列1枚ずつで変換して作る**。行列の中身は学習まかせ

なぜ変換が要るのか — 埋め込みそのままだと

- 変換などせず、==埋め込み x どうしの内積==を関連度にすればよさそうに見える：スコア $= XX^\top$

```
Code
1 import numpy as np
2
3 X = np.array([[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]])   # 3文字ぶんの埋め込み
4 print(X @ X.T)
```

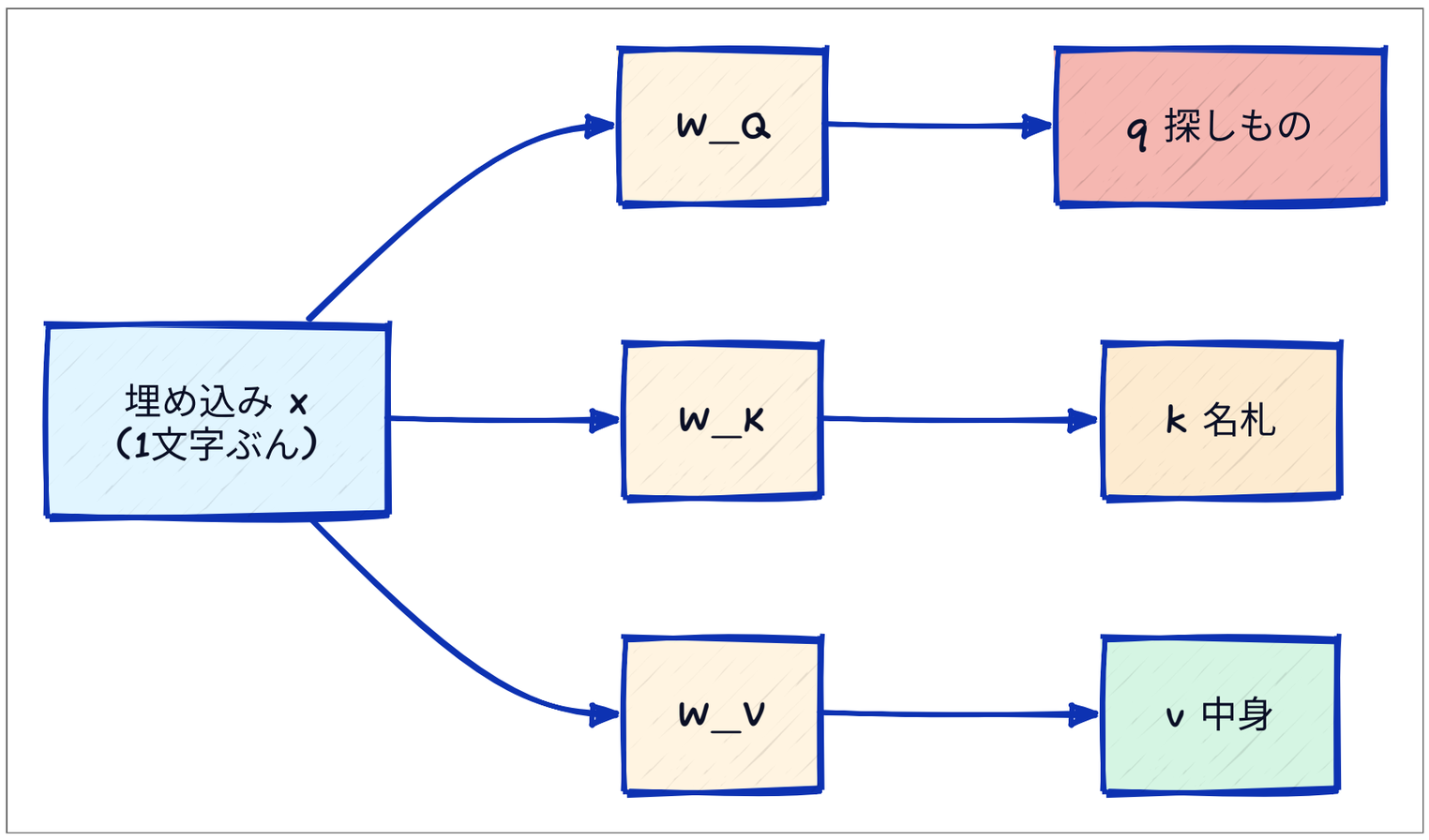
- 対角（自分×自分）が最有力 — 自分の埋め込みに一番似ているのは自分。**視線が自分に吸い付いて**、他人の情報がほとんど混ざらない
- しかも内積は**対称**：「AがBを見る強さ」＝「BがAを見る強さ」。だが言語の関係は非対称 — **「まだ」には「名前」が要るが、「名前」に「まだ」は要らない**
- 結論：**「私が探すもの」と「私が掲げる看板」は別物にしたい** — だから変換を挟む

q・k・v の作り方 — 行列を1枚ずつ

- 文字の埋め込み $\overrightarrow{x}$ から、3種類のベクトルを作る：

$$\overrightarrow{q} = \overrightarrow{x}\, W_Q \qquad \overrightarrow{k} = \overrightarrow{x}\, W_K \qquad \overrightarrow{v} = \overrightarrow{x}\, W_V$$

記号	名前	役割
Q	**クエリ**（query）	探しもの：「いま何の情報が欲しいか」
K	**キー**（key）	名札：「私はこういう情報を持っている」
V	**バリュー**（value）	中身：実際に混ぜられて運ばれる情報



- W_Q, W_K, W_V は勾配降下で学習されるただの行列 — とは、この3枚が育つこと

「どこを見るべきか」が学習される

全位置いっせいに

- 文が T 文字なら、埋め込みを縦に積んだ行列 X（T 行）に掛ければ**全文字ぶん一度に**変換できる — 行列積は**内積の一括処理機**

Code

```
import numpy as np

X = np.array([[1.0, 0.0],      # 3文字ぶんの埋め込み（おもちゃサイズ）
              [0.0, 1.0],
              [1.0, 1.0]])
Wq = np.array([[1.0, 0.0], [1.0, 1.0]])
Wk = np.array([[1.0, 1.0], [0.0, 1.0]])
Wv = np.array([[1.0, 0.0], [0.0, 1.0]])

Q, K, V = X @ Wq, X @ Wk, X @ Wv
scores = Q @ K.T                    # ① 全ペアの関連度を一撃で
print(scores)
```

- **`Q @ K.T`** の (i, j) 成分＝「i 文字目の探しもの」と「j 文字目の名札」の内積
- ==$T \times T$ の「視線の表」==が1回の行列積で出る

スケーリング — 内積は次元で膨らむ

① クイズ

ベクトルの次元 d を 2 → 64 に増やすと、ランダムな2本の内積の典型的な大きさは何倍になる？

Code

```
import numpy as np

rng = np.random.default_rng(0)
for d in [2, 64]:
    dots = [rng.standard_normal(d) @ rng.standard_normal(d) for _ in range(1000)]
    print(f"d={d:3d}: 内積の典型的な大きさ = {np.std(dots):.2f}")
```

- 足し合わせる項が d 個あるので $\sqrt{d}$ 倍で膨らむ（2→64 で約5.7倍）
- スコアが大きすぎると softmax が**一極集中**して、視線が1点に固まってしまう：

Code

```
s = np.array([8.0, 0.0, -8.0])              # d=64 スケールの生スコア
e = np.exp(s); print("そのまま:", (e / e.sum()).round(4))
e = np.exp(s / 8); print("÷√64  :", (e / e.sum()).round(4))
```

- だからスコアを $\sqrt{d}$ で割ってから softmax する — それだけの話

完成形 — 論文の式と3ステップ

- 2017年の論文 *Attention Is All You Need* の中心の式：

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

式の部品	3ステップでの正体
$QK^\top$	① 関連度を測る（全ペアの内積）
$\sqrt{d}$ **で割る**	①′ 膨らみ補正
$\mathrm{softmax}$	② 割合にする（行ごとに合計1）
$\cdots V$	③ 重み付き平均で混ぜる

- **行列積2回と softmax 1回**。世界を変えた式の中身は、先週の手計算

通しで動かす

Code

```
import numpy as np

def softmax_rows(S):
    e = np.exp(S - S.max(axis=1, keepdims=True))
    return e / e.sum(axis=1, keepdims=True)

X = np.array([[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]])
Wq = np.array([[1.0, 0.0], [1.0, 1.0]])
Wk = np.array([[1.0, 1.0], [0.0, 1.0]])
Wv = np.array([[1.0, 0.0], [0.0, 1.0]])

Q, K, V = X @ Wq, X @ Wk, X @ Wv
weights = softmax_rows(Q @ K.T / np.sqrt(2))    # 視線の表（各行の合計が1）
out = weights @ V                               # 各文字の「混ぜた結果」

print(weights.round(3))
print(out.round(3))
```

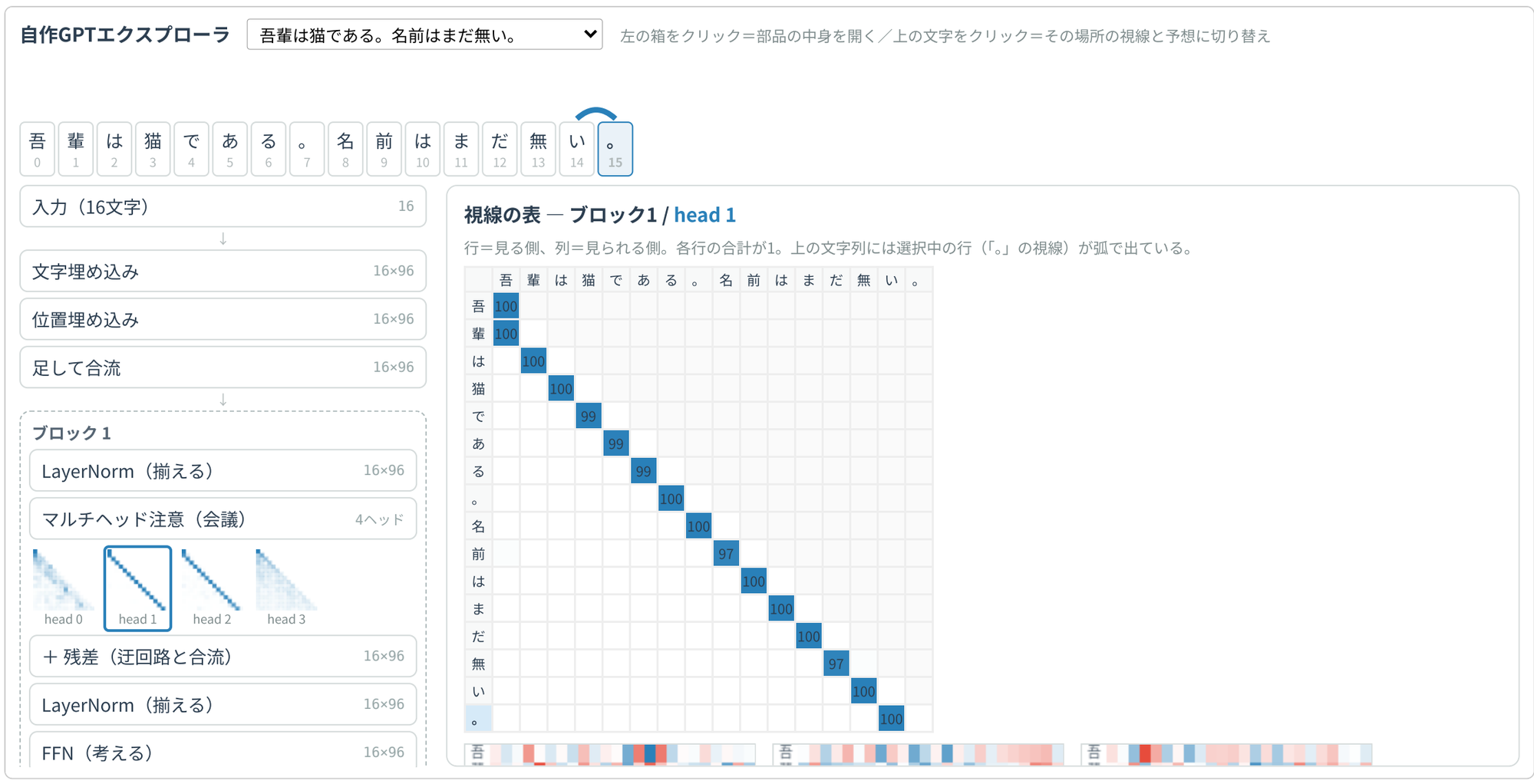
- 出力は**文字ごとに1本**：3文字入れたら、視線済みベクトルが3本返る
- どの文字も「自分の探しもの」で全員を見た — これを**自己注意**（**self-attention**）と呼ぶ

視線の表を絵にする

- 3文字目は3文字目自身を62%見ている、など**視線の配分が読める**
- この先、本物のモデルの視線もこの絵で覗く

本物の視線を先取りで覗く

この講義の最終形 —『吾輩は猫である』で訓練済みの自作GPT — の「視線の表」を、今日の道具だけで読んでみる。



0. 上の文字をクリックして、弧（＝その場所の視線）の飛び先を観察
1. この head 1 が「何を見る係」なのか、ひとことで言語化
2. 左の head 0・head 3 のサムネイルをクリックして、係の違いを確認（q・k・v の3枚のヒートマップも本物）

⚠ 気づいてしまった人へ

視線の表の**右上半分がきれいに空**になっている。今日の式 $\mathrm{softmax}(QK^\top/\sqrt{d})\,V$ のままでは、こうはならないはず — なぜか。次回の主役。

エクスプローラを別タブで開く

PyTorch 部品にしておく

- あとで組み立てやすいよう、部品をクラスにする。PyTorch の部品の型が **`nn.Module`**：==**`__init__`** に持ち物（層）、**`forward`** に計算手順==を書けば、**`sa(x)`** と呼べる部品になる

Code

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class SelfAttention(nn.Module):
    def __init__(self, n_embd, head_size):
        super().__init__()
        self.query = nn.Linear(n_embd, head_size, bias=False)   # Wq
        self.key   = nn.Linear(n_embd, head_size, bias=False)   # Wk
        self.value = nn.Linear(n_embd, head_size, bias=False)   # Wv

    def forward(self, x):                           # x: (文字数 T, n_embd)
        q, k, v = self.query(x), self.key(x), self.value(x)
        w = q @ k.T / k.shape[-1] ** 0.5            # ① 関連度 ＋ ①′ 補正
        w = F.softmax(w, dim=-1)                    # ② 割合にする
        return w @ v                                # ③ 混ぜる

torch.manual_seed(0)
sa = SelfAttention(n_embd=8, head_size=4)
x = torch.randn(5, 8)                               # 5文字・8次元
print(sa(x).shape)
```

- forward は実質3行。**`bias=False`** は「行列1枚」を式のとおりに書いただけ

Tweak 行列をいじって視線を操る



- **通しのコード** を書き換えて、視線の表の変化を見る

1	**`Wq` を `np.eye(2)`（単位行列）に**	探しものが埋め込みそのままになる。表はどう動く？
2	**`Wv` を `np.zeros((2, 2))` に**	視線はそのままなのに、出力だけ全部ゼロになる — QK と V の分業を確認
3	**スコアを `* 10` してから softmax**	視線が一極集中する（スケーリングの逆実験）

ここで観察

- **視線（どこを見るか）は Q と K だけ**で決まり、**運ばれる中身は V だけ**が決める。この分業が attention の設計の核心

まとめ

- 埋め込みそのままの内積では「自分に吸い付く・対称すぎる」— **探すものと看板を別物にする**のが変換の狙い
- Q（探しもの）・K（名札）・V（中身）は、埋め込みに**行列を1枚ずつ**掛けて作る — その行列が学習される
- 全ペアの関連度は **`Q @ K.T`** 一撃。$\sqrt{d}$ で膨らみを補正してから softmax
- $\mathrm{softmax}(QK^\top/\sqrt{d})\,V$ — 行列積2回＋softmax。もう読める
- 出力は文字ごとに1本の「視線済みベクトル」。自分たちで自分たちを見るので**自己注意**

現在の設計図　埋め込み → Q・K・V → ==$QK^\top/\sqrt{d}$ → softmax → V==

ⓘ 次回予告

実はこの仕組み、言語モデルに使うには**2つの大問題**を抱えている。 ひとつ：文字の順番を知らない。ふたつ：**未来がカンニングできてしまう**。

練習問題

以下の問題を解け（問題1は先に紙で答えてからコードで確認）。

問題1：形を追いかける

$T=5$ 文字、埋め込み $d=8$、**`head_size`** $=4$ のとき、次の行列の形（行数×列数）をそれぞれ答えよ。

1. Q、K、V
2. $QK^\top$（視線の表）
3. 出力 $\mathrm{softmax}(QK^\top/\sqrt{d})\,V$

`SelfAttention` の **`forward`** に **`print(q.shape)`** などを差し込んで検算せよ。

問題2：視線の表の行の合計

本編の「通しで動かす」コードで、**`weights`** の**各行の合計**を計算して1になることを確かめよ。列の合計はどうか。1にならないのはなぜか、1行で説明せよ。

解答例

▸ 解答例を見る